

SYSTEMS BLUEPRINT

MOVEO

Technical Architecture & Systems Blueprint

An enterprise-grade, multi-module ticket booking ecosystem featuring a Next.js client, Spring Boot backend, event-driven processing via Kafka, caching via Redis, real-time sync via WebSockets, and intelligent AI services.

PROJECT VERSION

v1.0.0 Stable

RELEASE DATE

May 31, 2026

DEVELOPED BY

Antigravity AI & Team

Platform Overview & Functional Blueprint

A detailed structural guide of the four application pillars.

Moveo is a comprehensive, next-generation booking ecosystem designed to offer a unified booking experience across movies, buses, and trains. It integrates visual premium seats charts, route selectors, and predictive tools, wrapping them into a single, cohesive client interface.

The platform bridges digital ticketing with high-fidelity, real-time synchronization, resolving the disconnect typical of legacy ticketing structures. Below are the functional specifications of the four pillars:

🎬 Movies Booking Module

The movies module is built around dynamic theater seating arrangements and screen synchronization. Showtimes are generated on-the-fly depending on cinema availability and region. Features include:

- Interactive seat selector grids representing precise premium and executive spatial orientations.
- Live-updating timer mechanisms during reservation locks (e.g., locking seats for 10 minutes to finish payments).
- Multiplex catalog filtering by audio language, sub-region, cinema chains, and 3D/4DX theater formats.

🚌 Buses Booking Module

The bus ticketing portal manages route schedules and amenities selections across major city routes. It addresses high-frequency schedules through a robust filtering UI.

- Seat layouts matching commercial single-deck, sleeper, and double-decker vehicle outlines.
- Intelligent boarding and dropping point lists mapped to interactive maps.
- Amenity filters supporting dynamic options such as Wi-Fi, AC, standard blankets, USB charging ports, and snacks.

Ø=P† Trains Booking Module

The train portal connects directly to national schedules, facilitating ticketing across several standard classes and regional systems.

- PNR search utilities returning passenger manifest details and active booking confirmations.
- Automatic allocation matching sleeper (SL), chair car (CC), and high-tier air-conditioned (3A, 2A, 1A) coach seats.
- Quota allocations support, handling Senior Citizen, Tatkal (rush bookings), Ladies, and General allocations.

Ø=Ü¬ Community & Forum Module

A social hub supporting ticket-sharing and route planning discussions. Users can rate films, share scenic bus routes, and discuss train timetables, fostering high-engagement community interactions.

CHAPTER 02

Technical Architecture & Core Stack

Deep dive into state, animations, data, and performance layers.

The application follows a decoupled multi-tier architecture, combining high-fidelity UI rendering on the client side with a robust Java/Spring Boot backend, accelerated by event pipelines and cache engines.

Ø<ß" Frontend Engine (Next.js & Animations)

The client application is built on React 19 and Next.js 16, utilizing modern server component paradigms and TypeScript. State is managed via Redux Toolkit, providing real-time data binding. Visual transitions are driven by industry-standard systems:

- GSAP (GreenSock Animation Platform): Used for complex timeline orchestrations, text reveals, and high-performance floating canvas elements.
- Lenis: Integrated smooth-scrolling engine maintaining fluid animation sync with hardware refresh rates.
- TailwindCSS v4: Provides the foundation for the visual design system, rendering fluid layouts, glassmorphic panels, and dark-mode tokens.

&™p Backend Engine (Spring Boot & MVC)

The core backend uses Java 17 and Spring Boot 3.x, structured to satisfy enterprise-grade separation of concerns across Entity, Repository, Service, DTO, and Controller classes. Security is managed via Spring Security with stateless JWT configurations, maintaining secure endpoints.

&j Distributed Messaging & Caching

To maintain consistency and handle seat conflicts under heavy loads, the platform relies on Redis and Apache Kafka:

- **Redis Cache:** Used for caching frequently accessed showtimes, bus schedules, and route paths. It maintains atomic seat locks to prevent double-booking conflicts.
- **Apache Kafka:** Event streaming backbone processing bookings data asynchronously, feeding statistics pipelines and dispatching tickets.
- **WebSockets:** Creates real-time full-duplex socket pipes syncing selected seats across users browsing the same movie screens.

CHAPTER 03

Intelligent AI Services & Engines

Integrating machine learning into ticket bookings, chat, and recommendations.

Moveo integrates specialized machine learning algorithms inside the `ai-service` workspace, transforming standard transactions into smart experiences:

Ø>Ý AI Chatbot (NLP Assistant)

An natural-language processor trained to interpret user search queries, fetch schedules, guide seat reservations, and answer cancellation policy questions. It integrates seamlessly into the Next.js chat layout, acting as a virtual assistant.

Ø=ÜÈ Fare Prediction Engine

A regression model processing seasonal demand, booking velocities, holidays, and route historical price trends. It projects fare fluctuations for buses and trains, warning users with indicators like: "Fares projected to rise by 15% in 3 days; book now."

Ø>Ýà Recommendation Engine

A collaborative filtering model tracking movie preferences, typical travel times, and frequently traveled routes. It surfaces customized promotions, featured grid entries, and seat recommendations, improving user engagement.

CHAPTER 04

DevOps, Infrastructure & Orchestration

A look into Nginx configurations, container clusters, and IAC scripts.

The DevOps pipeline maintains standard high availability, containerization, and infrastructure stability across regional staging and production environments.

Ø=Ý Nginx Load Balancing

Nginx manages the outer boundary, acting as a reverse proxy, load balancer, and TLS terminator. It forwards public `api` routes to Spring Boot services, redirects chatbot routes to Python fast-API services, and serves Next.js static bundles.

ðŸ“œ Containerization (Docker)

Every component is containerized using multi-stage Dockerfiles, building extremely light Alpine images that strip away compile-time development dependencies.

```
# Stage 1: Build Java Backend
FROM maven:3.8-openjdk-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Minimal Runtime
FROM openjdk:17-jdk-slim
COPY --from=build /app/target/moveo-api.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

&8p Orchestration (Kubernetes)

A robust K8s cluster deploys pods across regional availability zones. Custom Horizontal Pod Autoscalers (HPA) scale Spring Boot and frontend pods based on CPU and memory thresholds, handling peak booking rushes (e.g. Tatkal hours or Friday movie releases).

& p Infrastructure as Code (Terraform)

Terraform scripts provision standard development networks, VPCs, Kubernetes clusters, managed Redis caches, and database databases, assuring identical environments.

CHAPTER 05

API Specifications & Event Schemas

Comprehensive reference guides for REST endpoints, WebSockets, and event streaming.

The following tables provide the exact technical mappings required for integrating, consuming, and extending the Moveo ecosystem.

Ø=Üá Core REST API Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/auth/login	User sign-in & JWT dispatch	No
POST	/api/auth/signup	Create new user profile	No
GET	/api/movies/search	Search catalog & showtimes	No
POST	/api/movies/book	Initiate seat lock & transaction	Yes (JWT)
GET	/api/routes/buses	Query bus routes & amenities	No
GET	/api/routes/trains	Query train routes & schedules	No
POST	/api/bookings/history	Retrieve booking archives	Yes (JWT)

WebSocket Real-time Event Map

Event Name	Direction	Payload details	Usage Context
seat:lock	Client → Server	{ showtimeId, seats: [] }	Init seat selection lock
seat:lock:ok	Server → Client	{ status: "locked", expires: 600 }	Lock confirmed for 10 min
seat:update	Server →	{ showtimeId, lockedSeats: [] }	Sync seat colors for users
notify:ticket	Server → Client	{ bookingId, ticketUrl: "" }	Ticket generated successfully
chat:message	Client + Server	{ text: "", session: "" }	Interact with AI chatbot

Apache Kafka Event Streaming Topics

Topic Name	Producer Service	Consumer Service	Event Significance
bookings-log	Spring Boot Backend	AI Recommendation	Tracks booking logs for
notification-events	Spring Boot Backend	Email/Sms dispatch	Asynchronously triggers
fare-updates	Fare Predict Engine	Spring Boot Backend	Pushes dynamic fare

